

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Extensión e
implantación de
Solución Informática
para el aprendizaje
del lenguaje musical

Curso 2025/26

Alumno/a:

José Luis Garrido Castro

Director/es:

Javier Criado Rodríguez

AGRADECIMIENTOS

Me gustaría dar las gracias, en primer lugar, a la Escuela de Música de Huércal de Almería, al Real Conservatorio Profesional de Música de Almería y la Escuela de Música Manuel Sánchez Alonso de Canjáyar por formarme durante mis 17 últimos años, poner en mi vida este maravilloso arte que es la música y por prestarme su valiosa ayuda para este proyecto.

En segundo lugar, a mi familia ya que siempre me han apoyado a lo largo de toda mi vida y se que se sienten muy orgullosos de mis logros, en especial, los conseguidos a lo largo de este grado de Ingeniería Informática. Me han dado todo y no me cabe duda que siempre me lo darán.

Después, cabe mencionar a mis amigos que siempre estaban ahí para animarme en momentos malos y sacarme de casa cuando me hacía falta.

En particular, quería destacar a mi compañero Jesús, con el cual llevo coincidiendo desde el instituto, manteniendo una estrecha amistad que se ha fortalecido durante el grado pasando grandes momentos y ayudándonos de forma recíproca en lo que hemos necesitado cada uno.

A mi pareja que hasta hoy no ha quedado día que no me haya preguntado como me iba con todos mis proyectos (como este) y que siempre consigue mandarme un empujón y sacarme una sonrisa para seguir llevando todo a cabo.

Por último, a todos los profesores que me han ayudado a lo largo de todo el grado y que me han transmitido todos los conocimientos posibles así como permitirme desarrollar la gran cantidad de nuevas habilidades que he conseguido durante estos años.

ÍNDICE GENERAL

	Página
1. Contexto del proyecto y punto de partida	1
1.1. Marco teórico, investigación y estudio previo	1
1.2. Resultados del proyecto base y líneas de evolución	1
2. Objetivos del proyecto	3
3. Definición de la solución extendida	4
3.1. Análisis de la solución base y motivación de la extensión	4
3.2. Especificación formal de la solución extendida	5
3.3. Objetivos funcionales del sistema extendido	7
3.4. Diagrama UML de casos de uso del sistema extendido	7
3.5. Definición del diagrama de casos de uso	8
3.5.1. Casos de uso nuevos o modificados	9
3.5.1.1. Caso de uso: Importar usuarios y grupos desde archivo formateado	9
3.5.1.2. Caso de uso que sustituye Crear profesor: Gestionar profesores	9
3.5.1.3. Caso de uso que sustituye Crear alumno: Gestionar alumnos	9
3.5.1.4. Caso de uso existente ampliado: Gestionar grupos	9
3.5.1.5. Caso de uso existente ampliado: Interactuar con cuestionarios	10
3.5.2. Requisitos funcionales nuevos	10
3.6. Requisitos no funcionales nuevos	14
3.7. Tecnologías y arquitectura de la solución extendida	15
3.7.1. Pila tecnológica heredada del proyecto base	15
3.7.2. Tecnologías incorporadas en la solución extendida	16
3.7.2.1. Servicio de inferencia externa mediante Groq	16
3.7.2.2. Android Studio y entorno de construcción móvil	17
4. Planificación del proyecto	19
5. Desarrollo de la aplicación backend	21
5.1. Implementación	21

5.1.1. Operaciones lógicas (revisión y eliminación)	21
5.1.2. Importación masiva desde CSV (deformateo, validación y creación)	22
5.1.3. Servicio de IA con Groq (integración por API y capa de servicios)	36
5.2. Testing	37
6. Desarrollo de la aplicación frontend	39
6.1. Implementación	39
6.1.1. Exportación a Android con Capacitor	39
6.1.1.1. Ajuste de barras del sistema (zona superior e inferior)	39
6.1.1.2. Apertura de PDFs en Android (Filesystem + FileOpener)	40
6.1.1.3. Estilos específicos para móvil	41
6.1.1.4. Ajustes finales en Android Studio (iconos, splash, manifest)	41
6.2. Diseño	42
6.3. Testing	42
7. Plan de implantación	43
7.1. Adaptación tecnológica	43
7.2. Mejoras de seguridad	43
7.3. Implantación en un centro educativo	43
8. Conclusiones y líneas de trabajo futuro	44

ÍNDICE DE FIGURAS

3.1. Diagrama UML de casos de uso del sistema extendido	8
3.2. Tecnología de Inteligencia Artificial Groq.	16
3.3. Entorno de desarrollo y construcción móvil utilizado para la exportación a Android.	17
3.4. Arquitectura global de la aplicación (sin cambios respecto al proyecto base)	18
4.1. Planificación de las tareas principales en horas	20

ÍNDICE DE TABLAS

3.1. Caso de uso: Importar usuarios y grupos desde archivo formateado	9
3.2. Caso de uso: Gestionar profesores	9
3.3. Caso de uso: Gestionar alumnos	9
3.4. Caso de uso: Gestionar grupos	9
3.5. Caso de uso: Interactuar con cuestionarios	10
3.6. Requisito funcional: RF-31 Importar usuarios y grupos desde archivo formateado	10
3.7. Requisito funcional: RF-32 Revisar profesores	11
3.8. Requisito funcional: RF-33 Eliminar profesor	11
3.9. Requisito funcional: RF-34 Revisar alumnos	12
3.10. Requisito funcional: RF-35 Eliminar alumno	12
3.11. Requisito funcional: RF-36 Eliminar grupo	13
3.12. Requisito funcional: RF-37 Solicitar pista durante un cuestionario	14
3.13. Requisito no funcional: RNF-08 Portabilidad a Android	14

1 CONTEXTO DEL PROYECTO Y PUNTO DE PARTIDA

El presente proyecto se concibe como una extensión y evolución directa del Trabajo Fin de Grado desarrollado previamente, en el que se abordó el diseño, desarrollo y validación de una solución informática orientada a la enseñanza del lenguaje musical mediante una aplicación web con arquitectura cliente-servidor.

1.1 MARCO TEÓRICO, INVESTIGACIÓN Y ESTUDIO PREVIO

En el proyecto original se estableció, en primer lugar, un marco teórico que permitió contextualizar el lenguaje musical como disciplina educativa, analizando su papel dentro de la formación musical y las dificultades habituales asociadas a su aprendizaje y enseñanza. Este marco teórico puso de manifiesto una serie de problemáticas recurrentes, tales como la falta de recursos digitales específicos, la escasa adaptación de las herramientas existentes a las dinámicas reales del aula y la dificultad para realizar un seguimiento individualizado del alumnado en este ámbito.

A partir de esta base teórica, se llevó a cabo una investigación empírica apoyada en encuestas y entrevistas a profesores y alumnos de lenguaje musical, cuyo objetivo fue contrastar las problemáticas identificadas y detectar necesidades concretas desde la experiencia directa en entornos educativos reales. Paralelamente, se realizó un estudio basado en una revisión bibliográfica de trabajos previos y soluciones existentes, lo que permitió situar el proyecto dentro del estado del arte y analizar las limitaciones y oportunidades de las herramientas disponibles en la actualidad.

La combinación de marco teórico, investigación empírica y revisión bibliográfica proporcionó una base sólida para la definición de los requisitos funcionales y no funcionales del sistema, así como para la formulación de los objetivos del proyecto. Como resultado de este proceso, se diseñó e implementó una plataforma informática compuesta por un backend basado en una API REST y un frontend híbrido accesible desde distintos dispositivos.

1.2 RESULTADOS DEL PROYECTO BASE Y LÍNEAS DE EVOLUCIÓN

La aplicación desarrollada en el TFG original permite gestionar usuarios, grupos, material de clase, calificaciones, cuestionarios, tareas y comunicaciones internas, incorporando además mecanismos de autenticación, pruebas automatizadas, documentación técnica y procesos de despliegue continuo. Esta primera versión del sistema fue validada funcionalmente, demostrando su viabilidad técnica y su adecuación inicial a las necesidades detectadas.



No obstante, tanto las conclusiones del proyecto como la experiencia práctica derivada del uso de la aplicación evidenciaron que la solución implementada, aun siendo plenamente funcional, presenta margen de mejora y evolución. En particular, se identificaron oportunidades para refinar determinadas funcionalidades existentes, incorporar nuevas capacidades que amplíen el alcance de la aplicación y abordar de forma más profunda su implantación en entornos educativos reales. Estas líneas de evolución quedaron recogidas explícitamente como trabajo futuro en el TFG original.

En este contexto, el presente proyecto no pretende repetir ni reformular el marco teórico ni los estudios ya realizados, sino apoyarse en ellos como base validada para dar un paso adicional. El foco se desplaza desde el desarrollo inicial de la solución hacia su extensión funcional y su preparación para una implantación realista y sostenible, considerando aspectos técnicos, organizativos y de seguridad que resultan fundamentales en un escenario de uso continuado.

De este modo, este trabajo se plantea como una continuación natural y coherente del proyecto previo, orientada a consolidar la solución desarrollada y a sentar las bases necesarias para su evolución e implantación efectiva en centros educativos, manteniendo la alineación con la problemática detectada y con los principios pedagógicos que motivaron el proyecto original.

2 OBJETIVOS DEL PROYECTO

El objetivo general de este proyecto consiste en extender e implantar la solución informática desarrollada en el Trabajo Fin de Grado previo, llevándola a un nivel superior tanto en términos funcionales como de preparación para su uso en entornos educativos reales.

A partir de este objetivo general, se definen los siguientes subobjetivos específicos:

- **Mejorar y refinar funcionalidades ya existentes en la aplicación**, optimizando su usabilidad y adecuación a las necesidades detectadas en el contexto educativo.
- **Diseñar e incorporar nuevas funcionalidades innovadoras** que amplíen el alcance de la aplicación y aporten un valor añadido respecto a la versión inicial.
- **Aprovechar el diseño responsive y la arquitectura híbrida de la aplicación para facilitar su adaptación y despliegue en sistemas móviles**, permitiendo su ejecución en dispositivos Android y iOS sin necesidad de desarrollar una base de código independiente, y garantizando una experiencia de usuario coherente y optimizada en distintos tamaños de pantalla.
- **Definir un plan de implantación detallado** que contemple la adaptación tecnológica del sistema, la mejora de aspectos de seguridad, la gestión de archivos y datos, así como la viabilidad de su despliegue en centros educativos reales.

Estos objetivos se plantean manteniendo la coherencia con el proyecto original y aprovechando las conclusiones extraídas del proceso de investigación y desarrollo previo.

3 DEFINICIÓN DE LA SOLUCIÓN EXTENDIDA

Este complemento parte de la solución desarrollada en el proyecto base y se apoya en su definición previa. La aplicación original resolvía los procesos esenciales de gestión académica y comunicación asociados a la enseñanza del lenguaje musical, ofreciendo una plataforma con roles diferenciados (administrador, profesor y alumno), ramas de contenido, tareas y cuestionarios, sistema de mensajería y calificaciones.

Sin embargo, el propio enfoque del TFG —y especialmente sus conclusiones y líneas de trabajo futuro— asumía que una primera versión funcional no agota el problema: al llevar la solución a un entorno real aparecen necesidades operativas y oportunidades de mejora que no siempre resultan evidentes durante un desarrollo estrictamente académico. En particular, el uso prolongado en contextos educativos reales tiende a revelar fricciones relacionadas con la administración masiva, la gestión completa del ciclo de vida de usuarios y grupos y la experiencia de uso del alumnado en actividades evaluables.

En este punto, el objetivo de este complemento no es replantear la solución desde cero, sino evolucionarla de forma controlada. Se mantiene la arquitectura general, la filosofía de funcionamiento y la mayor parte de requisitos ya definidos en el documento principal, lo que se incorpora ahora se centra en:

- **Operatividad y administración real:** Reducir carga manual y facilitar tareas frecuentes en la práctica (altas masivas, etc.).
- **Gestión completa del ciclo de vida:** Incorporar operaciones de revisión y eliminación con comportamiento coherente y seguro.
- **Mejora de la experiencia del alumno:** Introducir mecanismos de apoyo durante el trabajo con cuestionarios para favorecer el aprendizaje sin romper la coherencia pedagógica.

Como en la definición del proyecto base, el proceso se plantea como un avance gradual desde un lenguaje natural todavía interpretativo hasta una definición progresivamente más objetiva. Primero se actualiza la especificación formal con las nuevas capacidades, después se refleja la ampliación en el diagrama de casos de uso, por último, se describen en detalle únicamente los casos de uso y requisitos afectados por los cambios (sin repetir lo ya definido en el TFG).

3.1 ANÁLISIS DE LA SOLUCIÓN BASE Y MOTIVACIÓN DE LA EXTENSIÓN

La solución original cubría de forma adecuada la operativa cotidiana de aula: creación de usuarios, creación y selección de grupos, publicación de material y tareas, comunicación con el alumnado, cues-

tionarios en la rama de teoría y gestión de calificaciones. Este conjunto de funcionalidades constituye una base sólida y coherente con la estructura pedagógica del lenguaje musical.

No obstante, al considerar una implantación real aparecen situaciones que requieren una respuesta más específica. Por ejemplo, la creación individual de usuarios funciona correctamente en grupos pequeños, pero se vuelve costosa cuando el número de alumnos aumenta o cuando se gestionan varios grupos simultáneos (escuelas con múltiples niveles, grupos por curso, etc.). De forma similar, en un entorno real es habitual que existan bajas, cambios de grupo, reorganizaciones o la necesidad de eliminar grupos completos cuando finaliza un curso o cuando se detectan altas erróneas, y estas operaciones deben dejar el sistema en un estado consistente.

Por último, en relación con el aprendizaje, la realización de cuestionarios tipo test (especialmente en teoría) tiende a producir bloqueos cuando el alumno no recuerda un concepto puntual. En una sesión real, el profesor suele ofrecer una pista o guía mínima para reconducir el razonamiento sin revelar la respuesta. Replicar parte de esa asistencia dentro de la herramienta puede mejorar la experiencia del alumno y reducir frustración, siempre que se haga con límites y con una intención pedagógica clara.

A partir de este análisis, se definen nuevas extensiones funcionales sobre la base ya existente.

3.2 ESPECIFICACIÓN FORMAL DE LA SOLUCIÓN EXTENDIDA

En esta sección se conserva la especificación formal descrita en el TFG original, incorporando únicamente las ampliaciones necesarias para reflejar la evolución del sistema. El objetivo es mantener continuidad, evitar redundancias y, al mismo tiempo, dejar completamente claro qué aporta este complemento. Las partes incorporadas a la especificación en esta fase estarán destacadas en color rojo.

El sistema permitirá el acceso mediante un mecanismo de inicio de sesión en el que podrán autenticarse tres tipos de usuarios: administrador, profesor y alumno. Cada usuario tendrá acceso únicamente a las funcionalidades correspondientes a su rol, sin posibilidad de utilizar las funciones asignadas a otros.

No existirá un proceso de registro. El administrador será creado de forma automática y, una vez dentro de la aplicación, tendrá la capacidad de crear las cuentas de los profesores manualmente. Para ello, deberá introducir los siguientes datos: nombre, primer apellido, segundo apellido y dirección de correo electrónico. Se realizará una comprobación de validez de los datos introducidos (formato correcto del nombre, de los apellidos y del correo electrónico) y una verificación de unicidad del correo electrónico en la base de datos.

En el proceso de creación, el sistema generará automáticamente un nombre de usuario a partir de la primera letra del nombre y de los apellidos. Por ejemplo, para un usuario denominado José Luis Garrido Castro, el identificador generado será jgc. En caso de que existan coincidencias con identificadores ya presentes en la base de datos, se añadirá un número consecutivo equivalente al total de usuarios con la misma raíz más uno, de modo que si ya existen 40 usuarios con identificadores iniciados en jgc, el nuevo identificador será jgc41. La contraseña será generada automáticamente mediante el uso de un servicio externo de generación de contraseñas seguras. Una vez creadas las credenciales, estas serán enviadas por correo electrónico al profesor.

Del mismo modo, el administrador podrá revisar la lista de profesores creados y eliminarlos. La eliminación de un profesor implicará la eliminación de toda la información asociada creada bajo su responsabilidad (material, tareas, cuestionarios, grupos y alumnado asociado a dichos grupos).

Además, el sistema permitirá al administrador realizar la creación de usuarios y grupos de manera automatizada. El fichero contendrá la información necesaria para dar de alta profesores y/o alumnos y para asociarlos a grupos, aplicando las mismas validaciones de datos y unicidad ya establecidas para la creación manual.

El profesor, a su vez, tendrá la capacidad de crear cuentas de alumnos mediante un procedimiento equivalente al descrito para la creación de profesores manual, **revisar y eliminar a sus alumnos (eliminando así sus registros de entregas y calificaciones)**.

Además, el profesor podrá crear grupos de clase, añadir alumnos a los mismos **y eliminar cualquiera de los grupos que haya creado borrando también de forma sistemática el material que tenga asociado**. Un alumno podrá pertenecer a varios grupos de forma simultánea.

También podrá enviar mensajes a unos alumnos o grupos completos determinados y podrá adjuntar las calificaciones generales del curso seleccionando a un alumno en concreto.

El sistema dispondrá de un área denominada administración en la que estarán disponibles las operaciones mencionadas anteriormente.

Fuera de esta sección, la aplicación contará con cuatro apartados denominados **Ritmo, Entonación, Audición y Teoría**. Cada apartado estará compuesto por dos secciones diferenciadas:

1. **Libro del curso:** Permitirá al profesor subir un único archivo en formato PDF correspondiente al grupo seleccionado. Si se sube un nuevo archivo, el anterior será sustituido. El profesor podrá eliminar o reemplazar el archivo en cualquier momento.
2. **Tareas a realizar:** Permitirá al profesor crear tareas asociadas a un grupo determinado. Cada tarea constará de una descripción y, opcionalmente, de un material de apoyo en formato PDF, MP3 o MP4. También se definirá una fecha límite de entrega. Las tareas podrán calificarse individualmente, modificarse y/o eliminarse en cualquier momento, pero no se eliminarán automáticamente al llegar la fecha de entrega. En el apartado de Teoría, las tareas podrán sustituirse (o no) por un cuestionario tipo test. Dicho cuestionario permitirá la creación de preguntas con opciones de respuesta, y cada pregunta podrá ir acompañada de un archivo en formato MP3.

Asimismo, los alumnos podrán acceder a los mismos cuatro apartados (Ritmo, Entonación, Audición y Teoría). En cada apartado se mostrarán las mismas dos secciones:

- En la sección Libro del curso, los alumnos podrán visualizar el archivo PDF correspondiente y descargarlo, pero no modificarlo.
- En la sección Tareas a realizar, los alumnos podrán consultar la lista de tareas disponibles y realizar las entregas correspondientes. Los formatos permitidos serán PDF, MP3 o MP4, y/o bien un comentario textual dentro de la propia aplicación. En el caso de los cuestionarios, el alumno podrá

acceder de forma específica a cada uno para completarlos **apoyándose, si lo desea, en solo una pista por cada pregunta que responda**. El cuestionario no se considerará entregado hasta que el alumno pulse expresamente el botón de envío.

Los alumnos y profesores dispondrán de un área específica de mensajes. En ella recibirán avisos cuando cuando se cree, se modifique, se califique (alumno) o se entregue (profesor) una tarea o cuestionario, cuando reciban una calificación general (mensajes del sistema) o un mensaje del profesor.

3.3 OBJETIVOS FUNCIONALES DEL SISTEMA EXTENDIDO

Los objetivos funcionales descritos en el TFG original se mantienen, ya que la esencia del sistema no cambia: facilitar la enseñanza del lenguaje musical con herramientas específicas de gestión y aprendizaje. No obstante, este complemento añade objetivos de evolución funcional centrados en la operatividad real y en la mejora de la experiencia del alumno:

- Permitir la administración masiva de usuarios y grupos mediante ficheros formateados, reduciendo la carga manual y los errores de alta.
- Completar la gestión del ciclo de vida de los usuarios (alumnos y profesores) agregando la posibilidad de revisarlos y eliminarlos y la de los grupos permitiendo eliminarlos.
- Incorporar mecanismos de apoyo al alumno durante la realización de cuestionarios para mejorar la experiencia de aprendizaje en actividades evaluables.

3.4 DIAGRAMA UML DE CASOS DE USO DEL SISTEMA EXTENDIDO

La ampliación funcional se refleja en el diagrama general de casos de uso del sistema. Para evitar confusión y mantener continuidad con el documento original, el diagrama se presentará con las nuevas incorporaciones destacadas visualmente mediante un código de color:

- **Rojo:** Caso de uso totalmente nuevo.
- **Azul:** Caso de uso existente sustituido (cambio de nombre y/o alcance).
- **Negro:** Caso de uso existente sin cambios de nombre (aunque pueda incorporar nuevos requisitos).

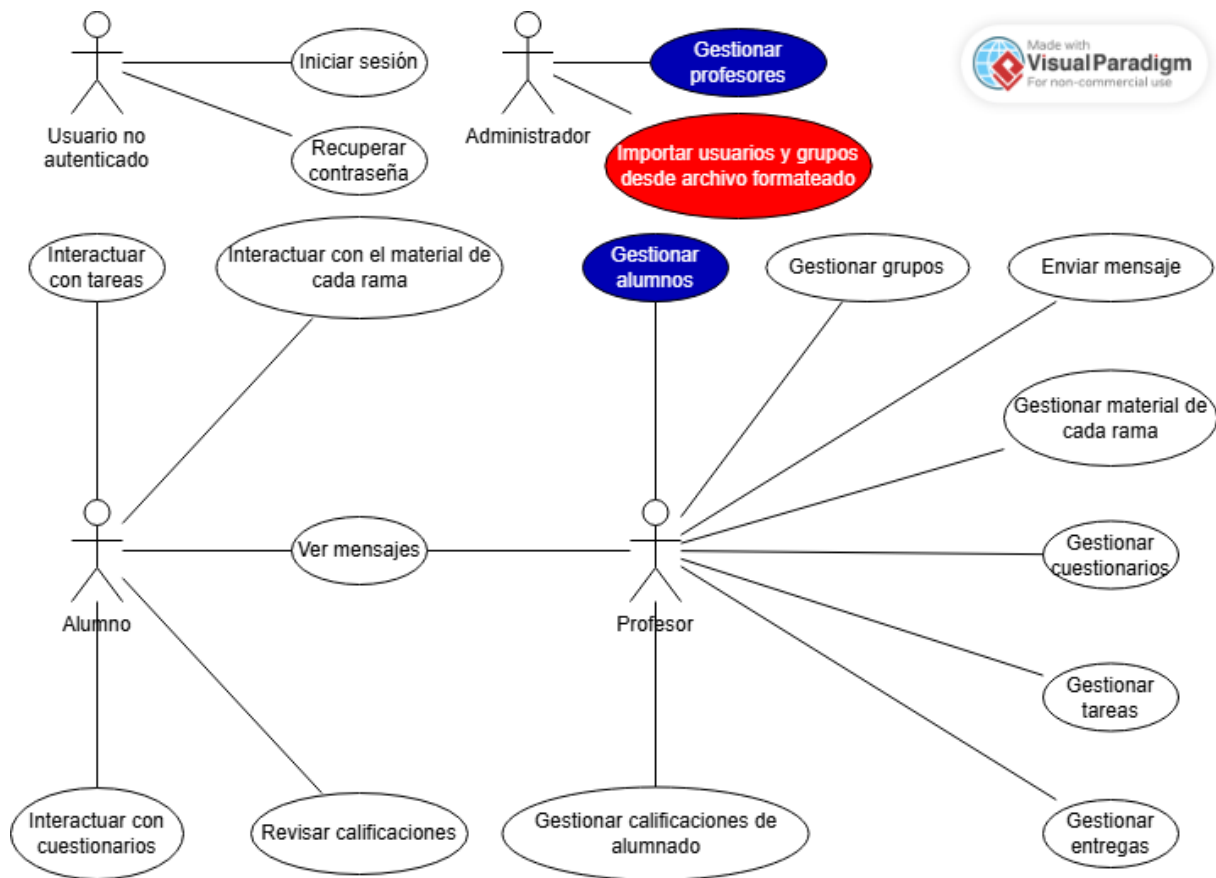


Figura 3.1: Diagrama UML de casos de uso del sistema extendido

3.5 DEFINICIÓN DEL DIAGRAMA DE CASOS DE USO

Como se detalló en el TFG, el diagrama ofrece una visión objetiva pero todavía abstracta del sistema. Para alcanzar un nivel operativo y verificable, se definen a continuación únicamente los casos de uso nuevos y aquellos existentes que se ven modificados por la incorporación de requisitos adicionales, manteniendo el resto de definiciones del proyecto base sin cambios. Cabe destacar que se ha mantenido el mismo código de color para el nombre del caso de uso y se ha usado el color rojo en el resto de campos de los casos de uso que no son totalmente nuevos para indicar en qué cambian.

3.5.1 Casos de uso nuevos o modificados

Caso de uso: Importar usuarios y grupos desde archivo formateado

Nombre	Importar usuarios y grupos desde archivo formateado
Descripción	Permitir al administrador crear usuarios y/o grupos mediante la carga de un fichero formateado, aplicando validaciones y creando relaciones de pertenencia según el contenido del archivo.
Actor	ACT-2
Requisitos funcionales	RF-31

Tabla 3.1: Caso de uso: Importar usuarios y grupos desde archivo formateado

Caso de uso que sustituye Crear profesor: Gestionar profesores

Nombre	Gestionar profesores
Descripción	Permitir al administrador crear, revisar y eliminar usuarios con el rol de profesor en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-2
Requisitos funcionales	RF-03 RF-04 RF-32 RF-33

Tabla 3.2: Caso de uso: Gestionar profesores

Caso de uso que sustituye Crear alumno: Gestionar alumnos

Nombre	Gestionar alumnos
Descripción	Permitir al profesor crear, revisar y eliminar usuarios con el rol de alumno en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-3
Requisitos funcionales	RF-05 RF-04 RF-34 RF-35

Tabla 3.3: Caso de uso: Gestionar alumnos

Caso de uso existente ampliado: Gestionar grupos

Nombre	Gestionar grupos
Descripción	Permitir al profesor crear nuevos grupos, revisar los que ha creado y elegir un grupo para trabajar con él o eliminarlo .
Actor	ACT-3
Requisitos funcionales	RF-06 RF-07 RF-08 RF-36

Tabla 3.4: Caso de uso: Gestionar grupos

Caso de uso existente ampliado: Interactuar con cuestionarios

Nombre	Interactuar con cuestionarios
Descripción	Permitir al alumno interactuar con los cuestionarios.
Actor	ACT-4
Requisitos funcionales	RF-18 RF-30 RF-37

Tabla 3.5: Caso de uso: Interactuar con cuestionarios**3.5.2 Requisitos funcionales nuevos**

En coherencia con el documento original, los identificadores continúan a partir del último requisito definido. A continuación se describen únicamente los nuevos requisitos y las ampliaciones necesarias de los casos de uso afectados.

RF-31	Importar usuarios y grupos desde archivo formateado
Descripción	Permitir al administrador crear usuarios y/o grupos mediante la carga de un fichero formateado, aplicando validaciones y creando relaciones de pertenencia según el contenido del archivo.
Precondición	El administrador debe estar autenticado y disponer de un fichero válido según el formato establecido.
Secuencia de pasos	1. El administrador accede a la opción de importación dentro del área de administración. 2. El administrador selecciona un fichero local para cargar. 3. El sistema valida el formato general del fichero (estructura, campos obligatorios y consistencia). 4. El sistema recorre los registros y valida datos (equivalente a RF-04 cuando aplique), incluyendo unicidad de correos y reglas de generación de credenciales. 5. El sistema crea los usuarios y grupos indicados si todos han sido validados. 6. El sistema informa del éxito.
Postcondición	Los usuarios y grupos válidos quedan registrados y listos para su uso
Excepciones	■ El fichero no cumple el formato establecido o falta información obligatoria. ■ Existen registros con correos duplicados o datos inválidos. ■ Problemas de conexión con la base de datos.

Tabla 3.6: Requisito funcional: RF-31 Importar usuarios y grupos desde archivo formateado

RF-32	Revisar profesores
Descripción	Permitir al administrador visualizar la lista de profesores registrados en el sistema y acceder a su información básica para su gestión.
Precondición	El administrador debe estar autenticado y tener profesores creados.
Secuencia de pasos	1. El administrador entra en la sección de profesores. 2. El sistema recupera y muestra el listado de profesores registrados.
Postcondición	El administrador puede identificar profesores gestionables y realizar acciones de creación o eliminación desde el mismo contexto.
Excepciones	■ No existen profesores registrados. ■ Problemas de conexión a la base de datos.

Tabla 3.7: Requisito funcional: RF-32 Revisar profesores

RF-33	Eliminar profesor
Descripción	Permitir al administrador eliminar un profesor del sistema, eliminando de forma todo su material asociado (tareas, cuestionarios, ramas, entregas y calificaciones), sus grupos y los alumnos creados bajo su responsabilidad, evitando referencias huérfanas o inconsistencias.
Precondición	El administrador debe estar autenticado y el profesor a eliminar debe existir.
Secuencia de pasos	1. El administrador accede a la administración de profesores y visualiza el listado. 2. El administrador selecciona un profesor y solicita su eliminación. 3. El sistema muestra confirmación. 4. El administrador confirma. 5. El sistema elimina al profesor y todos los datos relacionados con él. 6. El sistema informa del éxito.
Postcondición	El profesor deja de existir en el sistema y sus recursos asociados quedan gestionados de forma consistente.
Excepciones	■ El profesor no existe. ■ Problemas de conexión a la base de datos.

Tabla 3.8: Requisito funcional: RF-33 Eliminar profesor

RF-34	Revisar alumnos
Descripción	Permitir al profesor visualizar la lista de alumnos creados por él y acceder a su información básica para su gestión.
Precondición	El profesor debe estar autenticado y tener alumnos creados.
Secuencia de pasos	1. El profesor accede al área de administración y entra en la sección de alumnos. 2. El sistema recupera y muestra el listado de alumnos del profesor.
Postcondición	El profesor puede identificar a los alumnos gestionables y realizar acciones de creación o eliminación desde el mismo contexto.
Excepciones	■ No existen alumnos creados por el profesor. ■ Problemas de conexión a la base de datos.

Tabla 3.9: Requisito funcional: RF-34 Revisar alumnos

RF-35	Eliminar alumno
Descripción	Permitir al profesor eliminar un alumno creado por él, garantizando que el sistema gestiona de forma consistente la información asociada exclusivamente al alumno (calificaciones y entregas).
Precondición	El profesor debe estar autenticado y el alumno a eliminar debe haber sido creado previamente por el profesor.
Secuencia de pasos	1. El profesor accede al área de administración y visualiza el listado de alumnos gestionables. 2. El profesor selecciona un alumno y solicita su eliminación. 3. El sistema muestra un aviso de confirmación describiendo el alcance de la operación. 4. El profesor confirma la eliminación. 5. El sistema elimina el alumno, los datos exclusivos del alumno y las relaciones del alumno en el resto de registros que figure para mantener la consistencia. 6. El sistema informa del éxito.
Postcondición	El alumno deja de existir en el sistema y no es accesible desde la interfaz, a su vez, los datos asociados quedan en un estado consistente.
Excepciones	■ El alumno no existe o no ha sido creado por el profesor autenticado. ■ Problemas de conexión a la base de datos.

Tabla 3.10: Requisito funcional: RF-35 Eliminar alumno

RF-36	Eliminar grupo
Descripción	Permitir al profesor eliminar un grupo creado por él, gestionando de forma consistente todo el material asociado (tareas, cuestionarios, ramas, entregas y calificaciones).
Precondición	El profesor debe estar autenticado y el grupo a eliminar debe haber sido creado previamente por el profesor.
Secuencia de pasos	1. El profesor visualiza su listado de grupos y selecciona el grupo a eliminar. 2. El profesor solicita la eliminación del grupo. 3. El sistema solicita confirmación, indicando que se trata de una eliminación irreversible. 4. El profesor confirma. 5. El sistema elimina el grupo y todos los datos relacionados con el grupo menos los usuarios, a los cuales les elimina la relación para mantener consistencia. 6. El sistema informa del éxito.
Postcondición	El grupo deja de existir en el sistema y no puede seleccionarse, además, no quedan recursos asociados accesibles de forma inconsistente.
Excepciones	■ El grupo no existe o no ha sido creado por el profesor autenticado. ■ Problemas de conexión a la base de datos.

Tabla 3.11: Requisito funcional: RF-36 Eliminar grupo

RF-37	Solicitar pista durante un cuestionario
Descripción	Permitir al alumno solicitar solo una pista asociada a una pregunta concreta durante la realización de un cuestionario, recibiendo una orientación limitada que facilite el razonamiento sin revelar la respuesta.
Precondición	El alumno debe estar autenticado, haber accedido a un cuestionario activo y encontrarse en el proceso de respuesta.
Secuencia de pasos	1. El sistema muestra las preguntas del cuestionario. 2. El alumno solicita una pista para la pregunta que tenga dudas. 3. El sistema presenta una pista contextual (orientación), sin mostrar directamente la opción correcta. 4. El alumno continúa respondiendo a la pregunta. 5. El sistema permite finalizar y entregar el cuestionario como en RF-30.
Postcondición	El alumno recibe una pista y puede continuar la realización de la entrega bajo las reglas de RF-30.
Excepciones	■ El cuestionario no está activo o el alumno no tiene acceso al mismo. ■ El servicio de generación de pista no está disponible. ■ Problemas de conexión a la base de datos.

Tabla 3.12: Requisito funcional: RF-37 Solicitar pista durante un cuestionario

3.6 REQUISITOS NO FUNCIONALES NUEVOS

El TFG original definía un requisito de portabilidad (RNF-04) orientado a asegurar que la aplicación pudiera utilizarse en distintos dispositivos y entornos. En este complemento, esa portabilidad se concreta en un objetivo más exigente y verificable: la exportación efectiva a un entorno móvil Android, aprovechando el diseño híbrido y responsive ya adoptado.

RNF-08	Portabilidad a Android
Descripción	El sistema deberá poder ejecutarse como aplicación instalada en dispositivos Android, manteniendo las funcionalidades esenciales y una experiencia de usuario consistente.
Justificación	Se da continuidad a RNF-04 (Portabilidad) del proyecto base, materializando la ejecución en un entorno móvil real para reforzar accesibilidad y facilitar su implantación en contextos educativos.
Criterio de cumplimiento	La aplicación se exporta y ejecuta en Android mediante herramientas de construcción móvil, pudiendo autenticarse, navegar por ramas, consultar materiales y realizar acciones principales sin degradación funcional significativa.

Tabla 3.13: Requisito no funcional: RNF-08 Portabilidad a Android

3.7 TECNOLOGÍAS Y ARQUITECTURA DE LA SOLUCIÓN EXTENDIDA

La solución extendida mantiene la arquitectura global cliente-servidor basada en una API REST y basándose en el patrón MVC (Modelo-Vista-Controlador) idéntica en su planteamiento al proyecto base. Esta continuidad tecnológica no es accidental: la pila original proporciona un equilibrio adecuado entre modularidad, escalabilidad y facilidad de mantenimiento, lo que permite incorporar nuevas capacidades sin necesidad de reestructurar el sistema ni introducir deuda técnica.

3.7.1 Pila tecnológica heredada del proyecto base

El proyecto se apoya en un conjunto de tecnologías consolidadas que cubren de forma eficaz las necesidades de una aplicación educativa con múltiples roles, gestión de contenidos multimedia y comunicación bidireccional:

- **Ionic + Angular (Frontend).** Se mantiene un frontend híbrido desarrollado con Ionic y Angular, que permite construir una interfaz responsive y reutilizable tanto en entorno web como móvil. Angular aporta una estructura basada en componentes, servicios y módulos que facilita la organización del código, la reutilización de lógica y la separación entre presentación y lógica de negocio, mientras que Ionic proporciona componentes visuales optimizados para dispositivos táctiles y móviles.
- **Node.js + Express (Backend).** El backend continúa implementándose mediante un servidor Node.js con Express, exponiendo una API REST encargada de centralizar la lógica de negocio, la validación de datos y la aplicación de reglas de acceso. Esta combinación ofrece un entorno ligero y flexible, adecuado para aplicaciones con múltiples operaciones CRUD y consumo intensivo de recursos multimedia.
- **MongoDB (Persistencia).** Se mantiene una base de datos documental MongoDB para almacenar los datos que requieren de persistencia. El modelo no relacional facilita la evolución del esquema de datos y permite representar relaciones entre entidades educativas sin penalizar la flexibilidad, algo especialmente relevante en un sistema que crece de forma incremental. También se hace uso de MongoDB Atlas, un servicio gestionado en la nube que ofrece una solución escalable y segura.
- **JWT (Autenticación y autorización).** La autenticación se sigue gestionando mediante JSON Web Tokens, permitiendo un control claro de sesiones y permisos en función del rol del usuario. Este enfoque se adapta bien a una arquitectura REST y evita mantener estado de sesión en el servidor.
- **Swagger / OpenAPI (Documentación).** La documentación de la API se mantiene mediante Swagger, proporcionando una especificación clara y verificable de los endpoints disponibles, útil tanto para pruebas como para mantenimiento futuro.
- **Testing (Jest/Supertest y Jasmine/Karma).** Se conserva el uso de Jest y Supertest para pruebas del backend, y Jasmine/Karma para el frontend, permitiendo validar de forma automatizada el comportamiento del sistema y reducir regresiones durante la evolución del proyecto.

- **Integración y despliegue continuo (Render, GitHub Actions y Pages).** Se continua ejecutando el flujo de integración y despliegue continuo en GitHub Actions para automatizar el testing y el despliegue, que será realizado en Render para el backend y GitHub Pages para el frontend.

3.7.2 Tecnologías incorporadas en la solución extendida

La extensión del sistema introduce dos tecnologías nuevas que amplían las capacidades técnicas de la aplicación sin modificar su arquitectura base.

Servicio de inferencia externa mediante Groq

Se incorpora un servicio de inferencia externa utilizando Groq como proveedor, haciendo uso de su API pública para el envío de peticiones desde el backend del sistema. En este contexto, Groq actúa como un servicio externo capaz de procesar *prompts* de inteligencia artificial y devolver respuestas generadas a partir de modelos de lenguaje, sin que dichos modelos formen parte de la infraestructura propia de la aplicación.

Desde un punto de vista tecnológico, la utilización de la API de Groq implica la integración de un servicio HTTP externo que recibe solicitudes estructuradas, incluyendo el contenido del *prompt*, y devuelve una respuesta generada en formato estándar (por ejemplo, JSON). Esta aproximación permite tratar el sistema de inferencia como un recurso remoto, consumido de forma similar a cualquier otro servicio web.

El uso de una API de inferencia externa presenta varias implicaciones técnicas relevantes:

- Permite externalizar el procesamiento asociado a modelos de lenguaje, evitando la necesidad de desplegar, mantener o escalar infraestructura específica para inferencia.
- Facilita el control del flujo de peticiones, ya que las llamadas a la API pueden regularse, validarse y encapsularse desde el backend.
- Aísla la lógica de generación de contenido del resto del sistema, permitiendo modificar el proveedor o el modelo utilizado sin afectar a la arquitectura interna.

Desde el punto de vista del diseño software, la integración mediante una API externa refuerza el desacoplamiento del sistema, ya que el servicio de inferencia se consume como una dependencia externa claramente delimitada, manteniendo la coherencia con la arquitectura por capas adoptada en el proyecto.



Figura 3.2: Tecnología de Inteligencia Artificial Groq.

Android Studio y entorno de construcción móvil

La segunda tecnología incorporada es Android Studio, utilizado como entorno de construcción, empaquetado y validación de la aplicación en dispositivos Android. Su inclusión se apoya en el carácter híbrido del frontend, que permite reutilizar la misma base de código web para generar una aplicación instalable.

Desde el punto de vista tecnológico, el uso de Android Studio implica:

- La integración con la cadena de herramientas oficial de Android para compilación, firma y despliegue de aplicaciones.
- La generación de un contenedor nativo que encapsula la aplicación híbrida, permitiendo su ejecución como aplicación instalada.
- El acceso a herramientas de depuración, emulación y análisis propias del ecosistema Android.

Este enfoque no introduce una nueva base de código ni una bifurcación tecnológica, sino que extiende el alcance del sistema aprovechando el diseño responsive existente. De este modo, la portabilidad se aborda como una extensión técnica natural del frontend híbrido, sin duplicar lógica ni comprometer la consistencia del sistema.



Figura 3.3: Entorno de desarrollo y construcción móvil utilizado para la exportación a Android.

En conjunto, las tecnologías incorporadas refuerzan la arquitectura existente, ampliando las capacidades del sistema desde un punto de vista estrictamente técnico y manteniendo los principios de modularidad, reutilización y separación de responsabilidades definidos en el proyecto base.

A nivel conceptual, la arquitectura global permanece idéntica. Se incluye a continuación el diagrama original para facilitar la lectura del documento sin obligar a consultar constantemente el TFG.

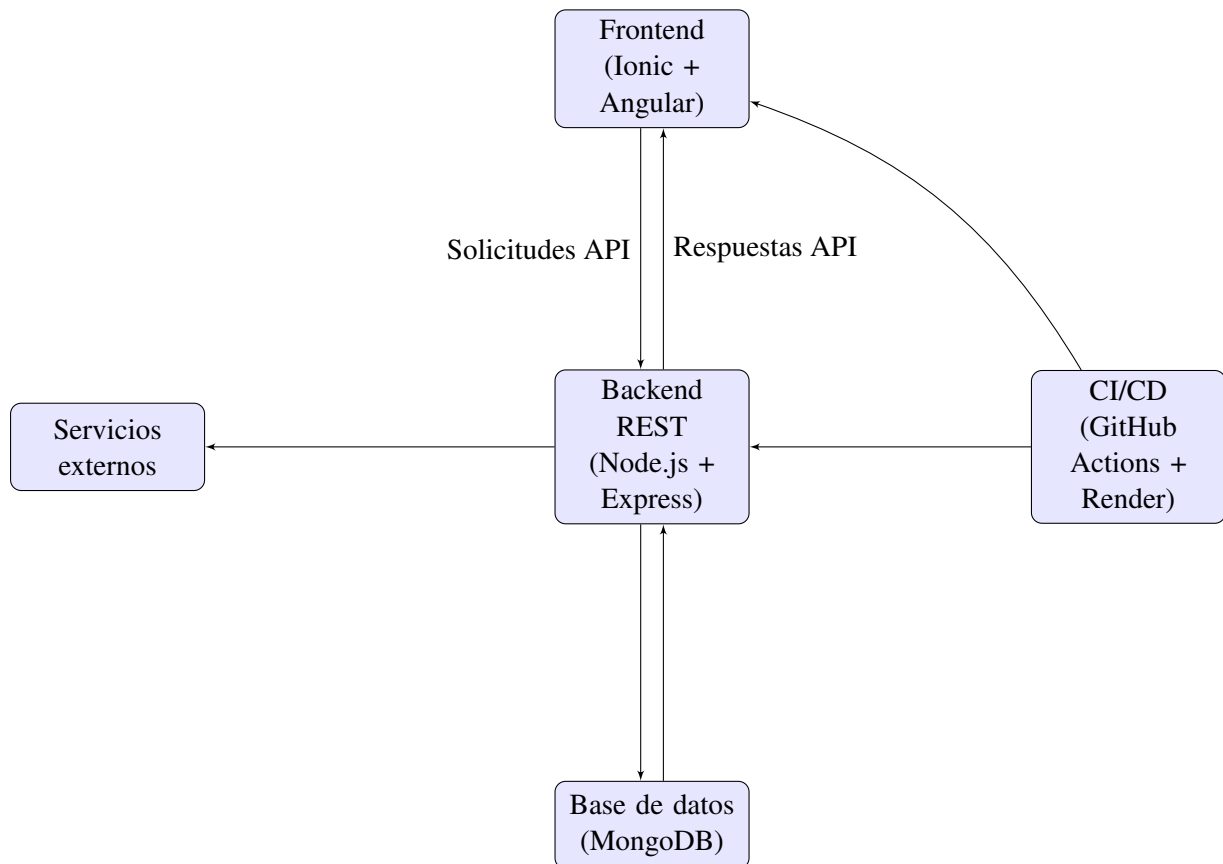


Figura 3.4: Arquitectura global de la aplicación (sin cambios respecto al proyecto base)

Con esta definición, la solución extendida queda descrita con el mismo enfoque del proyecto base: continuidad en arquitectura y metodología, y ampliación funcional bien acotada, verificable y orientada a la implantación real.

4 PLANIFICACIÓN DEL PROYECTO

La planificación de este complemento se concibe como una extensión directa de la planificación definida en el proyecto original, manteniendo sus mismos criterios generales de organización, seguimiento y gestión del tiempo. Dado que la base del sistema ya se encuentra diseñada e implementada, no resulta necesario replantear la estructura global del cronograma, sino ampliarla de forma coherente para incorporar las nuevas tareas derivadas de la evolución funcional del proyecto.

En este contexto, la planificación del complemento se centra en integrar las actividades adicionales asociadas a la mejora de funcionalidades, la incorporación de nuevas capacidades y la preparación del sistema para su implantación, respetando la secuencia lógica y la continuidad del trabajo previamente realizado. De este modo, se garantiza que la evolución del proyecto se produce de forma ordenada y alineada con los objetivos ya establecidos.

El diagrama Gantt de planificación reutiliza la estructura del cronograma original, incorporando las nuevas tareas y sub-tareas de manera diferenciada mediante un código de color, lo que permite identificar claramente qué actividades pertenecen al proyecto base y cuáles corresponden específicamente a este complemento. Esta representación facilita la comprensión del alcance real de la extensión sin introducir redundancias en la documentación.

En conjunto, la planificación del complemento actúa como un marco de referencia que guía la ejecución de esta fase de evolución del proyecto, manteniendo la flexibilidad propia del contexto académico y asegurando la coherencia con el trabajo previamente desarrollado.

Por último, también se añade la planificación de las tareas principales en horas.

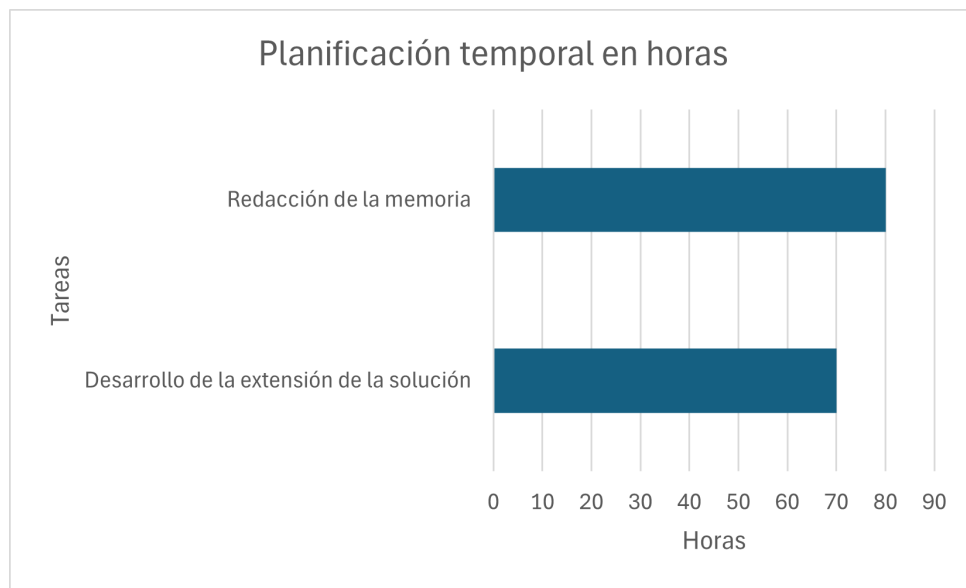


Figura 4.1: Planificación de las tareas principales en horas

5 DESARROLLO DE LA APLICACIÓN BACKEND

El backend de este complemento se apoya directamente en la base implementada en el TFG original, manteniendo la misma pila tecnológica: **Node.js** como entorno de ejecución y **Express** como framework para exponer una **API REST**. A nivel de organización interna, se conserva la separación de responsabilidades típica del proyecto base, con una estructura inspirada en **MVC adaptado a backend** (sin vistas): **modelos** para persistencia y esquema de datos, **controladores** para la lógica de operaciones y **rutas** como capa de entrada a los endpoints.

En esta fase del complemento, la mayor diferencia no está en “hacer un CRUD nuevo”, sino en reforzar operaciones de uso real (revisiones, eliminaciones y altas masivas) que requieren más validación, limpieza de relaciones y control de errores. Además, aparece una pieza nueva dentro del backend: **servicios**, usados para encapsular integraciones externas como la comunicación con la **API de Groq**.

5.1 IMPLEMENTACIÓN

5.1.1 Operaciones lógicas (revisión y eliminación)

Las operaciones de revisión (listados y consultas) reutilizan el mismo patrón del proyecto base: filtrado por rol, respuesta en JSON y acceso a través de controladores. En cambio, las eliminaciones se han ampliado para contemplar el efecto real en otras colecciones (por ejemplo: al eliminar un grupo se eliminan/limpian tareas, cuestionarios, entregas y calificaciones asociadas, y se elimina la relación en los alumnos).

Se incluye un ejemplo representativo (el resto de eliminaciones sigue una estructura muy similar):

```
// Ejemplo representativo: eliminar un grupo y limpiar datos asociados
async function eliminarGrupo(groupId, profesorId) {

  // 1) Verificar que el grupo existe y pertenece al profesor autenticado
  const grupo = await Grupo.findOne({
    _id: groupId,
    profesor: profesorId
  });

  if (!grupo) {
    throw new Error('Grupo no encontrado o sin permisos');
  }
}
```

```
// 2) Eliminar contenido asociado al grupo
await Tarea.deleteMany({ grupo: groupId });
await Cuestionario.deleteMany({ grupo: groupId });

// 3) Eliminar entregas y calificaciones asociadas
await Entrega.deleteMany({ grupo: groupId });
await Calificacion.deleteMany({ grupo: groupId });

// 4) Eliminar la relación del grupo en alumnos
await Alumno.updateMany(
  { grupos: groupId },
  { $pull: { grupos: groupId } }
);

// 5) Eliminar el documento principal
await Grupo.deleteOne({ _id: groupId });

return { ok: true };
}
```

5.1.2 Importación masiva desde CSV (deformateo, validación y creación)

La importación desde CSV introduce un flujo distinto al alta individual: se trabaja con datos externos que pueden venir incompletos, repetidos o con referencias inconsistentes. Por eso, el proceso se estructura en dos fases:

- **Validación previa completa:** se parsea el CSV y se recorren todas las filas acumulando errores. Si existe cualquier error, no se crea nada.
- **Creación en transacción:** si el CSV es válido, se crea todo dentro de una transacción de Mongoose para evitar estados parciales.

Además, se aplica una regla del sistema clave en este complemento: **un alumno pertenece a un único profesor**. Esto se comprueba antes de crear los grupos: si un grupo de un profesor intenta incluir un alumno asignado a otro profesor, se detiene la importación.

A continuación se incluye el fragmento completo solicitado. Está intencionadamente **muy comentado** y con **saltos de línea extra** para minimizar que se salga del margen en PDF.

```
const generarUsername = async (baseUsername) => {
```



```
// -----  
//  Generación del username final a partir de una base de 3 letras  
//  - Se busca cuántos usuarios existen ya con ese prefijo  
//  - Si hay coincidencias, se añade un sufijo numérico  
// -----  
  
if (baseUsername.length !== 3) {  
  return {  
    error: 'La base para el username debe ser de 3 letras.'  
  };  
}  
  
const regex = new RegExp(`^${baseUsername}`);  
  
const count = await Usuario.countDocuments({  
  username: regex  
});  
  
return {  
  username: count > 0  
    ? `${baseUsername}${count}`  
    : baseUsername  
};  
};  
  
const normalize = (s) =>  
  (s || '')  
    .toString()  
    .trim()  
    .toLowerCase()  
  
  // -----  
  //  Normalización Unicode para eliminar acentos:  
  //  Ej.: "José" -> "Jose"  
  // -----  
  .normalize('NFD')  
  .replace(/[\u0300-\u036f]/g, '')  
  
  // -----  
  //  Se eliminan caracteres que no sean letras (a-z)  
  //  Ej.: espacios, guiones, números, etc.  
  // -----  
  .replace(/[^a-z]/g, '');
```



```
const deriveBaseUsername3 = ({
  nombre,
  apellido1,
  apellido2,
  email
}) => {

  // -----
  // A partir de nombre/apellidos/email, se deriva una base de
  // 3 letras para username (antes de aplicar el sufijo numérico)
  // -----

  const n = normalize(nombre);
  const a1 = normalize(apellido1);
  const a2 = normalize(apellido2);

  const em = email
    ? normalize(email.split('@')[0])
    : '';

  let base = '';

  // Iniciales típicas: nombre + apellidos
  if (n) base += n[0];
  if (a1) base += a1[0];
  if (a2) base += a2[0];

  // Si no alcanza 3 caracteres, se rellena con letras extra
  if (base.length < 3) {

    const extra =
      a1.slice(1)
      + n.slice(1)
      + em;

    for (const ch of extra) {
      if (base.length >= 3) break;
      if (ch) base += ch;
    }
  }

  // En último caso, se fuerza a 3 caracteres
  if (base.length < 3) {
    base = (base + 'xxx').slice(0, 3);
  }
}
```



```
    return base.slice(0, 3);
};

const isLikelyObjectId = (v) =>
  typeof v === 'string'
  && /^[a-zA-Z0-9]{24}$/.test(v);

exports.importarDesdeCSV = async (fileBuffer) => {

  // -----
  // 1) Parseo del CSV
  // - columns: true -> usa la primera fila como cabecera
  // - trim: true -> elimina espacios
  // -----

  let records;

  try {
    records = parse(fileBuffer, {
      columns: true,
      skip_empty_lines: true,
      trim: true
    });
  } catch (e) {
    return {
      status: 400,
      body: {
        error: 'CSV inválido o mal formateado.',
        details: e.message
      }
    };
  }

  if (!Array.isArray(records) || records.length === 0) {
    return {
      status: 400,
      body: { error: 'El CSV está vacío.' }
    };
  }

  // -----
  // 2) Preparación de estructuras
```

```
// - errors: lista de errores por fila
// - usersRows: filas tipo "usuario"
// - groupsRows: filas tipo "grupo"
// - seenRefs / seenEmails: evitar duplicados dentro del CSV
// -----

const errors = [];

const usersRows = [];
const groupsRows = [];

const seenRefs = new Set();
const seenEmails = new Set();

// -----
// 3) Validación fila a fila
// - Se recorren todas las filas y se registran errores
// - NO se crea nada todavía
// -----

for (let i = 0; i < records.length; i++) {

  const r = records[i];

  // +2: porque la fila 1 es cabecera
  // y queremos mostrar número humano (base 1)
  const rowNum = i + 2;

  const tipo = (r.tipo || '').toLowerCase();

  // ----- Validación de "ref" -----
  if (!r.ref) {
    errors.push(`Fila ${rowNum}: 'ref' es obligatorio.`);
  }

  if (r.ref && seenRefs.has(r.ref)) {
    errors.push(
      `Fila ${rowNum}: ref duplicado '${r.ref}'`
    );
  }

  if (r.ref) seenRefs.add(r.ref);
}
```



```
// ----- Tipos permitidos -----
if (tipo === 'usuario') {

  // Rol: alumno o profesor
  const rol = (r.rol || '').toLowerCase();

  if (!['alumno', 'profesor'].includes(rol)) {
    errors.push(
      `Fila ${rowNum}: rol inválido '${r.rol}'.`
    );
  }

  // Campos obligatorios de usuario
  for (const f of [
    'nombre',
    'apellido1',
    'apellido2',
    'email'
  ]) {
    if (!r[f]) {
      errors.push(
        `Fila ${rowNum}: '${f}' es obligatorio para usuario.`
      );
    }
  }

  // Email único dentro del CSV
  if (r.email) {
    const em = r.email.toLowerCase();

    if (seenEmails.has(em)) {
      errors.push(
        `Fila ${rowNum}: email duplicado en el CSV '${r.email}'.`
      );
    }

    seenEmails.add(em);
  }

  // Reglas específicas de alumno
  if (rol === 'alumno' && !r.profesor_ref) {
    errors.push(
      `Fila ${rowNum}: alumno requiere 'profesor_ref'.`
    );
  }
}
```

```
    usersRows.push({
      ...r,
      rol
    });

    } else if (tipo === 'grupo') {

      // Campos obligatorios de grupo
      for (const f of [
        'nombre_grupo',
        'profesor_ref',
        'alumnos_ref'
      ]) {
        if (!r[f]) {
          errors.push(
            `Fila ${rowNum}: '${f}' es obligatorio para grupo.`
          );
        }
      }

      groupsRows.push(r);

    } else {

      errors.push(
        `Fila ${rowNum}: tipo inválido '${r.tipo}'. ` +
        `Use 'usuario' o 'grupo'.`
      );
    }
  }

  // -----
  // 4) Separación por referencias del CSV
  // - Map(ref -> fila profesor)
  // - Map(ref -> fila alumno)
  // -----

  const profRefRows = new Map();
  const alumRefRows = new Map();

  for (const u of usersRows) {
    if (u.rol === 'profesor') profRefRows.set(u.ref, u);
    if (u.rol === 'alumno')   alumRefRows.set(u.ref, u);
  }
```

```
// -----  
// 5) Validación: cada alumno debe apuntar a un profesor válido  
// - Puede ser una ref dentro del CSV  
// - O un ObjectId (si el profesor ya existe en BD)  
// -----  
  
for (const a of alumRefRows.values()) {  
  
  const pref = a.profesor_ref;  
  
  const ok =  
    profRefRows.has(pref)  
    || isLikelyObjectId(pref);  
  
  if (!ok) {  
    errors.push(  
      `Alumno ref='${a.ref}': profesor_ref '${pref}' ` +  
      `no existe en CSV ni parece ObjectId.`  
    );  
  }  
}  
  
// -----  
// 6) Validación de grupos  
// - profesor_ref válido  
// - alumnos_ref no vacío  
// - refs de alumnos válidas  
// - regla de negocio:  
//   un alumno no puede pertenecer a otro profesor  
// -----  
  
for (const g of groupsRows) {  
  
  const pref = g.profesor_ref;  
  
  const profesorOk =  
    profRefRows.has(pref)  
    || isLikelyObjectId(pref);  
  
  if (!profesorOk) {  
    errors.push(  
      `Grupo ref='${g.ref}': profesor_ref '${pref}' ` +  
      `no existe en CSV ni parece ObjectId.`  
    );  
  }  
}
```

```
}

// alumnos_ref se separa con '|'
const alumnoRefs = (g.alumnos_ref || '')
  .split('|')
  .map((s) => s.trim())
  .filter(Boolean);

if (alumnoRefs.length === 0) {
  errors.push(
    `Grupo ref='${g.ref}': debe tener al menos un alumno ` +
    `en alumnos_ref.`
  );
}

// 6.1) Validación: cada alumno existe en CSV o es ObjectId
for (const ar of alumnoRefs) {

  const existeEnCsv = alumRefRows.has(ar);
  const pareceId    = isLikelyObjectId(ar);

  if (!existeEnCsv && !pareceId) {
    errors.push(
      `Grupo ref='${g.ref}': alumno '${ar}' ` +
      `no existe en CSV ni parece ObjectId.`
    );
  }
}

// 6.2) Regla de negocio: alumno pertenece a un profesor único
for (const ar of alumnoRefs) {

  if (alumRefRows.has(ar)) {

    const alumno = alumRefRows.get(ar);

    if (alumno.profesor_ref && alumno.profesor_ref !== pref) {

      errors.push(
        `Regla negocio: Grupo ref='${g.ref}' ` +
        `(profesor_ref='${pref}') incluye alumno ref='${ar}' ` +
        `que pertenece a profesor_ref='${alumno.profesor_ref}'.`
      );
    }
  }
}
}
```

```
}

// Si hay errores, se devuelve antes de crear nada
if (errors.length > 0) {
  return {
    status: 400,
    body: {
      error: 'Errores de validación en el CSV.',
      errors
    }
  };
}

// -----
// 7) Creación dentro de transacción (Mongoose session)
// - Si algo falla, se aborta y no queda nada a medias
// -----

const session = await mongoose.startSession();

// Se guardan usuarios creados para enviar credenciales al final
const usersToEmail = [];

try {

  await session.startTransaction();

  // --- 7.1) Crear profesores ---
  const profesoresRows = [...profRefRows.values()];

  const profesoresData = profesoresRows.map((p) => ({
    nombre: p.nombre,
    apellido1: p.apellido1,
    apellido2: p.apellido2,
    email: p.email,

    // Se deriva baseUsername de 3 letras
    baseUsername: deriveBaseUsername3(p)
  }));

  const createdProfesoresByRef = new Map();

  if (profesoresData.length > 0) {
```



```
const res = await exports.addUsuarios(
  profesoresData,
  'profesor',
  null,
  session,
  { sendEmails: false }
);

if (res.status !== 201) {
  throw new Error(
    res.body?.error || 'Error creando profesores'
  );
}

const created = res.body;

profesoresRows.forEach((row, idx) => {
  createdProfesoresByRef.set(row.ref, created[idx]);
  usersToEmail.push(created[idx]);
});
}

// --- 7.2) Agrupar alumnos por profesor ---
const alumnosPorProfesor = new Map();

for (const [aref, a] of alumRefRows.entries()) {

  // profesor_ref puede ser ObjectId o ref del CSV
  const profesorId = isLikelyObjectId(a.profesor_ref)
    ? a.profesor_ref
    : String(
        createdProfesoresByRef
          .get(a.profesor_ref)
          ._id
      );

  const data = {
    nombre: a.nombre,
    apellido1: a.apellido1,
    apellido2: a.apellido2,
    email: a.email,
    baseUsername: deriveBaseUsername3(a),

    // Guardamos ref del CSV para mapear el creado luego
```

```
    __csv_ref: aref
  };

  if (!alumnosPorProfesor.has(profesorId)) {
    alumnosPorProfesor.set(profesorId, []);
  }

  alumnosPorProfesor.get(profesorId).push(data);
}

// --- 7.3) Crear alumnos y mapear por ref ---
const createdAlumnosByRef = new Map();

for (const [profesorId, alumnosData] of alumnosPorProfesor.entries()) {

  const res = await exports.addUsuarios(
    alumnosData.map((x) => ({
      nombre: x.nombre,
      apellido1: x.apellido1,
      apellido2: x.apellido2,
      email: x.email,
      baseUsername: x.baseUsername
    })),
    'alumno',
    profesorId,
    session,
    { sendEmails: false }
  );

  if (res.status !== 201) {
    throw new Error(
      res.body?.error || 'Error creando alumnos'
    );
  }

  const created = res.body;

  alumnosData.forEach((a, idx) => {
    createdAlumnosByRef.set(a.__csv_ref, created[idx]);
    usersToEmail.push(created[idx]);
  });
}

// --- 7.4) Crear grupos ---
```

```
for (const g of groupsRows) {

  const profesorId = isLikelyObjectId(g.profesor_ref)
    ? g.profesor_ref
    : String(
      createdProfesoresByRef
        .get(g.profesor_ref)
        ._id
    );

  const alumnoRefs = (g.alumnos_ref || '')
    .split('|')
    .map((s) => s.trim())
    .filter(Boolean);

  const alumnoIds = alumnoRefs.map((ar) =>
    isLikelyObjectId(ar)
      ? ar
      : String(
        createdAlumnosByRef
          .get(ar)
          ._id
      )
  );

  const res = await grupoController.crearGrupo(
    g.nombre_grupo,
    profesorId,
    alumnoIds,
    session
  );

  if (res.status !== 201) {
    throw new Error(
      res.body?.error || 'Error creando grupo'
    );
  }
}

// --- 7.5) Commit total ---
await session.commitTransaction();
session.endSession();

// -----
```



```
// 8) Envío de correos con credenciales
// - se realiza fuera de transacción
// - si falla un correo, no invalida lo creado
// -----

for (const user of usersToEmail) {

  const emailResult = await emailController.enviarEmailCredenciales(
    user.email,
    user.username,
    user.password
  );

  if (emailResult.message !== 'Correo enviado correctamente.') {

    console.error(
      `Error al enviar correo a ${user.email}: `
      + `${emailResult.error || emailResult.message}`
    );
  }
}

// Respuesta final de éxito
return {
  status: 201,
  body: {
    message: 'Importación CSV completada.',
    created: {
      profesores: profRefRows.size,
      alumnos: alumRefRows.size,
      grupos: groupsRows.length
    }
  }
};

} catch (e) {

  // Si algo falla, se aborta toda la transacción
  try {
    await session.abortTransaction();
  } catch (_) {}

  session.endSession();

  return {
```

```
    status: 400,  
    body: {  
      error: 'La importación falló y se revirtió.',  
      details: e.message  
    }  
  };  
}  
};
```

5.1.3 Servicio de IA con Groq (integración por API y capa de servicios)

En el proyecto base, la lógica se repartía principalmente entre **rutas**, **controladores** y **modelos**. En este complemento se introduce además una **capa de servicios** para encapsular integraciones externas. Groq se integra mediante su **API pública**, y en la práctica el backend envía **prompts** (texto estructurado con instrucciones y contexto) y recibe una respuesta generada. Separarlo en un servicio evita mezclar esta comunicación externa con la lógica interna del sistema.

```
// services/aiHint.service.js  
const Groq = require('groq-sdk');  
  
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });  
  
async function generarPistaTeoria({  
  preguntaTexto,  
  posiblesRespuestas,  
  recursoAudicion  
) {  
  
  if (!process.env.GROQ_API_KEY) {  
    throw new Error('GROQ_API_KEY no está configurada');  
  }  
  
  const opciones = Array.isArray(posiblesRespuestas)  
    ? posiblesRespuestas  
      .map((r, i) => `${i + 1}) ${r?.texto ?? ''}`)  
      .join('\n')  
    : '';  
  
  const userContent =  
    `Genera UNA pista breve para ayudar a responder esta pregunta tipo test sin revelar la  
  
Pregunta:  
${preguntaTexto}`
```

```
Opciones:
${opciones}

${recursoAudicion ? 'Hay un recurso de audición asociado.' : ''}

Requisitos:
- 1 o 2 frases máximo
- No digas cuál es la opción correcta
- No des la solución ni pasos completos
- No menciones "IA" ni "modelo";

const completion = await groq.chat.completions.create({
  model: 'llama-3.3-70b-versatile',
  messages: [
    { role: 'system', content: 'Eres un tutor de música. Das pistas breves y seguras' },
    { role: 'user', content: userContent }
  ],
  temperature: 0.2,
  max_tokens: 90
});

const hint =
  completion?.choices?.[0]?.message?.content?.trim();

if (!hint) throw new Error('No se pudo generar pista');

return hint
  .replace(/^[``\s]+|["'\s]+$/g, '')
  .trim();
}

module.exports = { generarPistaTeoria };
```

5.2 TESTING

Para el backend se mantiene el enfoque del TFG original con **Jest** y **Supertest**, centrado en validar endpoints y respuestas HTTP en un entorno controlado. En este complemento, cobra especial importancia asegurar que:

- una importación inválida devuelve errores sin crear datos,
- una importación válida crea lo esperado,



- y operaciones sensibles (como eliminaciones) responden correctamente según permisos.

6 DESARROLLO DE LA APLICACIÓN FRONTEND

El frontend del complemento se mantiene sobre la pila del proyecto base: **Angular + Ionic**. Angular aporta la estructura modular basada en **componentes**, **servicios** y **módulos**, mientras que Ionic proporciona componentes visuales pensados para interfaz táctil y ejecución híbrida.

A nivel de patrón, el desarrollo sigue el esquema habitual de Angular (frecuentemente descrito como un **MVC/MVVM práctico**): los **componentes** gestionan estado y renderizado de la vista, los **servicios** encapsulan lógica reutilizable y comunicación HTTP, y la “capa modelo” se representa mediante interfaces/DTOs y estructuras que viajan entre frontend y backend.

En este complemento, aunque se han añadido pantallas y funcionalidades, el cambio más visible y relevante en frontend es la **exportación real a Android** y los ajustes necesarios para que la aplicación se comporte bien como app instalada.

6.1 IMPLEMENTACIÓN

6.1.1 Exportación a Android con Capacitor

La exportación a móvil se realiza con **Capacitor**, que actúa como puente entre el proyecto web y el proyecto nativo Android. A nivel práctico:

- genera un contenedor Android para ejecutar la app como instalada,
- permite usar plugins para acceder a capacidades del dispositivo,
- y facilita ajustes que en navegador (PC) no son necesarios.

Ajuste de barras del sistema (zona superior e inferior)

En móvil pueden aparecer solapes con la barra superior o la zona inferior de navegación. Una forma de resolverlo es detectar plataforma nativa y aplicar estilos específicos.

```
// Ejemplo orientativo: detectar ejecución nativa (Capacitor)
import { Capacitor } from '@capacitor/core';

const isNative = Capacitor.isNativePlatform();

if (isNative) {
```



```
document.body.classList.add('native-platform');  
}
```

```
/* Ejemplo orientativo: padding extra en móvil para evitar solapes */  
.native-platform ion-content {  
  --padding-top: 16px;  
  --padding-bottom: 16px;  
}
```

Apertura de PDFs en Android (Filesystem + FileOpener)

En PC, incrustar un PDF con `iframe` suele funcionar bien, pero en Android el `WebView` puede dar problemas. La solución aplicada consiste en:

- detectar que se está en móvil,
- mostrar un botón *Abrir PDF*,
- guardar temporalmente el archivo en el dispositivo,
- y abrirlo con el visor nativo del sistema.

```
// Ejemplo orientativo: abrir PDF en visor nativo Android  
import { Capacitor } from '@capacitor/core';  
import { Filesystem, Directory } from '@capacitor/filesystem';  
import { FileOpener } from '@capacitor-community/file-opener';  
  
async function abrirPdfEnMovil(  
  nombreFichero,  
  pdfBase64  
) {  
  
  // Solo se ejecuta en app instalada (Android/iOS)  
  if (!Capacitor.isNativePlatform()) return;  
  
  // 1) Guardar el PDF en Documents (o caché si prefieres)  
  const writeRes = await Filesystem.writeFile({  
    path: nombreFichero,  
    data: pdfBase64,  
    directory: Directory.Documents  
  });  
}
```

```
// 2) Abrir el PDF con el visor nativo
await FileOpener.open({
  filePath: writeRes.uri,
  contentType: 'application/pdf'
});
}
```

Estilos específicos para móvil

Además de los cambios anteriores, se han aplicado ajustes de CSS para mejorar usabilidad en pantallas pequeñas (márgenes, tamaños de letra, botones a ancho completo, etc.). Un ejemplo típico es el uso de @media:

```
/* Ejemplo orientativo de ajustes visuales para móvil */
@media (max-width: 768px) {

  ion-button {
    width: 100%;
    margin-bottom: 12px;
  }

  .content-wrapper {
    padding: 12px;
  }

  .card-title {
    font-size: 1.05rem;
  }
}
```

Ajustes finales en Android Studio (iconos, splash, manifest)

Una vez compilado el proyecto Android, ciertos recursos visuales se terminan de ajustar desde **Android Studio**, como:

- iconos definitivos,
- pantalla de carga (*splash*),
- y referencias en el *manifest* si procede.

6.2 DISEÑO

En este complemento, las capturas de diseño se plantean **desde móvil** (Android) para evidenciar justo lo explicado en implementación: comportamiento instalado, ausencia de solapes, apertura de PDFs con visor nativo y coherencia visual en pantalla pequeña.

6.3 TESTING

El frontend mantiene el enfoque del TFG original con **Jasmine** y **Karma**. En esta fase, el objetivo principal es asegurar que los componentes siguen comportándose como se espera tras cambios y ajustes para móvil, y que los servicios que consumen la API siguen devolviendo datos de forma consistente.

A continuación se incluye un ejemplo genérico (sustituible por tus tests reales):

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { MiComponentePage } from '../mi-componente.page';

describe('MiComponentePage', () => {

  let component: MiComponentePage;
  let fixture: ComponentFixture<MiComponentePage>;

  beforeEach(async () => {

    await TestBed.configureTestingModule({
      declarations: [MiComponentePage],
    }).compileComponents();

    fixture = TestBed.createComponent(MiComponentePage);
    component = fixture.componentInstance;

    fixture.detectChanges();
  });

  it('debe crearse correctamente', () => {
    expect(component).toBeTruthy();
  });

  it('debe inicializar el estado por defecto', () => {
    expect(component).toBeDefined();
  });

});
```

7 PLAN DE IMPLANTACIÓN

Este apartado constituye uno de los ejes principales del proyecto y describe el conjunto de acciones necesarias para implantar la solución en un entorno educativo real.

7.1 ADAPTACIÓN TECNOLÓGICA

Se identifican los cambios necesarios para migrar la solución hacia tecnologías más robustas y adecuadas para un entorno productivo, incluyendo mejoras en la gestión de archivos, el almacenamiento de datos y la escalabilidad del sistema.

7.2 MEJORAS DE SEGURIDAD

Aunque no todas las mejoras de seguridad propuestas se implementan directamente en esta fase, se detallan las modificaciones necesarias para cumplir con buenas prácticas, tales como la gestión segura de credenciales, el almacenamiento cifrado de contraseñas y la protección de datos sensibles.

7.3 IMPLANTACIÓN EN UN CENTRO EDUCATIVO

Se describe el proceso de implantación de la aplicación en un centro educativo real, incluyendo la adaptación del flujo de trabajo del profesorado y del alumnado.

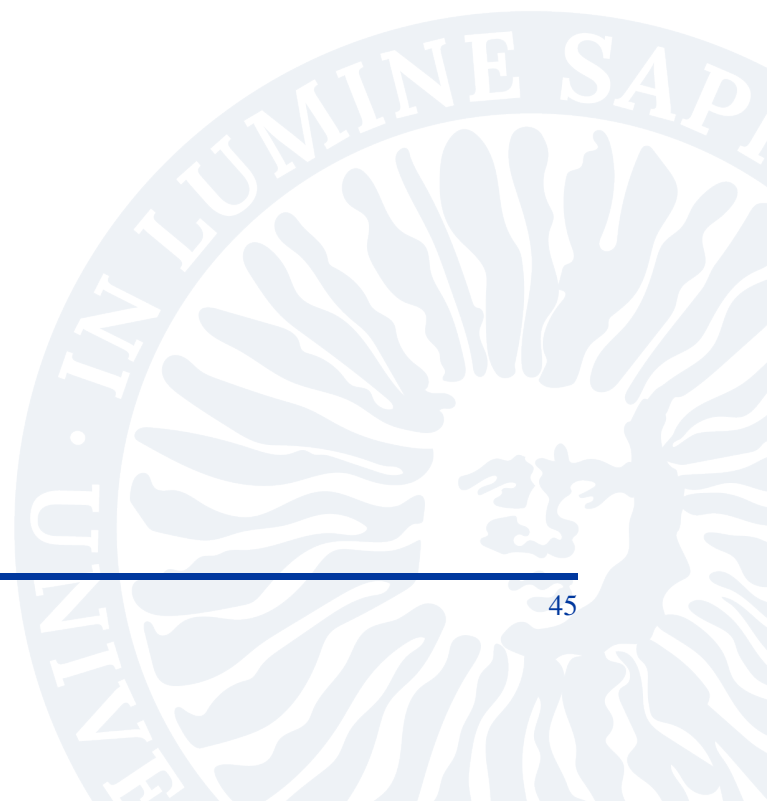
8 CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURO

El proyecto desarrollado demuestra la viabilidad de evolucionar una solución informática educativa desde una primera versión funcional hacia una plataforma más madura y preparada para su implantación real.

Las mejoras introducidas y el plan de implantación definido sientan las bases para futuras ampliaciones, entre las que se incluyen la incorporación de funcionalidades más avanzadas, la mejora continua de la seguridad y la adaptación del sistema a distintos contextos educativos.

Este trabajo refuerza el carácter práctico y aplicado del proyecto original, consolidando la solución desarrollada como una herramienta con potencial real en el ámbito de la enseñanza del lenguaje musical.

BIBLIOGRAFÍA



El proyecto ha consistido en el estudio e investigación de las diferentes herramientas, tecnologías o cualquier aspecto que pudiera relacionar la asignatura de lenguaje musical con la ingeniería informática, así como la definición, desarrollo y despliegue de una solución informática para resolver la carencia de recursos informáticos en el proceso de aprendizaje del lenguaje musical.

The project has consisted of the study and research of different tools, technologies, and any other aspects that could relate the subject of music theory to computer engineering, as well as the definition, development, and deployment of a software solution aimed at addressing the lack of digital resources in the process of learning music theory.

